ToggleMaster · Fase 4

Guia único de gravação

Fala, tela e comando no mesmo lugar, cena por cena

10	21 min	4	25 min
cenas	roteiro + 4 de margem	prints obrigatórios	limite da FIAP

DIGA
a fala, em ordem

FAÇA
onde clicar e o que deve aparecer

PRINT
capture naquele momento

★
bloco exigido pela FIAP

Fonte das falas: docs/gravacao/roteiro-conteudo.json · comandos: PASSO_A_PASSO_POR_CENA.md · enunciado FIAP p. 5–6

Monte a mesa antes do REC

Abas na ordem das cenas; terminais com uma tarefa cada. Docker Desktop fechado.

ABAS DO BRAVE

1	GitHub · README	cena 01, 02, 05, 10
2	GitHub · pasta terraform/	cena 02, 03, 05
3	Actions · run do auth-service no merge	cena 02
4	ArgoCD · Applications (localhost:8080)	cena 03
5	Grafana · ToggleMaster - Visao geral	cena 04, 07
6	Grafana · Explore → Loki	cena 05
7	Grafana · Alert rules → FlagService5xxAlto	cena 08, 09
8	New Relic · Service map	cena 06
9	New Relic · Distributed tracing	cena 06
10	PagerDuty · Incidents	cena 08, 09
11	Slack · #incidentes-tech-challenge	cena 08, 09
12	Actions · workflow self-healing	cena 09

JANELAS

A — Terminal A
log do pod trafego · cenas 07–09

B — Terminal B
pods do flag-service · cenas 07–09

C — Terminal C
comandos · cenas 03, 06, 07

D — Terminal D
túneis · minimizado, fora do vídeo

L — Lens
Nodes e Pods · cenas 03 e 05

Fora do vídeo: .env, Secrets, senhas e páginas de chaves

Terminal D · abra os três túneis

Cada túnel roda num laço que o reabre sozinho em 2 s se a conexão cair.

1 · portas livres?

→ três linhas "livre"

```
for p in 8080 3000 8004; do netstat -ano | grep -q ":$p .*LISTENING" && echo "porta $p OCUPADA" || echo "porta $p livre"; done
```

2 · ArgoCD (8080)

```
(while true; do kubectl -n argocd port-forward svc/argocd-server 8080:80; sleep 2; done) > /tmp/tunel-argocd.log 2>&1 &
```

3 · Grafana (3000)

```
(while true; do kubectl -n observability port-forward svc/grafana 3000:80; sleep 2; done) > /tmp/tunel-grafana.log 2>&1 &
```

4 · evaluation (8004)

```
(while true; do kubectl -n togglemaster port-forward svc/evaluation-service 8004:8004; sleep 2; done) > /tmp/tunel-evaluation.log 2>&1 &
```

5 · conferência, 5 s depois

→ três linhas começando com 200; minimize a janela D sem fechar

```
for u in http://127.0.0.1:8080 http://127.0.0.1:3000/api/health http://127.0.0.1:8004/health; do echo "$(curl -s -o /dev/null -w '%{http_code}' -m 5 $u) $u"; done
```

Terminais A, B e C · tráfego, pods e logins

As senhas vão direto para a área de transferência, sem aparecer na tela.

Terminal A · tráfego

→ flag-service /flags -> 200 a cada segundo

```
kubectl logs -f trafego -n togglemaster --tail=5
```

Terminal B · pods do flag-service

→ 1 pod Running

```
kubectl get pods -n togglemaster -l app=flag-service -w
```

Terminal C · senha do ArgoCD (admin, localhost:8080)

```
kubectl -n argocd get secret argocd-initial-admin-secret -o
jsonpath='{.data.password}' | base64 -d | clip
```

Terminal C · senha do Grafana (admin, localhost:3000)

```
kubectl -n observability get secret grafana-admin -o jsonpath='{.data.admin-password}'
| base64 -d | clip
```

Terminal C · última conferência, depois rode clear

→ AGE maior que 10 min (cooldown de 600 s)

```
kubectl get pods -n togglemaster -l app=flag-service
```

01 Apresente o problema

00:00–01:00 · GitHub · README

DIGA

Este é o ToggleMaster, uma plataforma de feature flags com cinco microsserviços. A Fase 4 evolui a base das fases anteriores para responder a três perguntas: o sistema está saudável, onde uma requisição falhou e como reagimos ao incidente?

Mantivemos Terraform, CI/CD com verificações de segurança e GitOps. Agora acrescentamos métricas, logs, traces e uma tentativa automática de recuperação.

Vou demonstrar o ambiente funcionando e depois provocar uma falha controlada para acompanhar a resposta completa.

FAÇA · ONDE → O QUE APARECE

Aba 1 · topo do README
→ *título e "cinco microsserviços"*

Aba 1 · role até "O que foi reaproveitado e o que a Fase 4 acrescentou"
→ *coluna da base ao lado da coluna da Fase 4*

Aba 1 · role até "Arquitetura de observabilidade e resposta" e "Serviços e arquitetura da base"
→ *diagrama e tabela dos cinco serviços*

02 Mostre a base automatizada

01:00–03:00 · GitHub · código + Actions

DIGA

A infraestrutura está dividida em três camadas Terraform, com estados separados.

O código fica no Git; o state fica no backend S3 e relaciona os recursos declarados aos recursos existentes.

Os cinco serviços possuem pipelines que reutilizam verificações por linguagem. Aqui vemos build e testes, análise do código e das dependências.

No pull request, a imagem também é construída e verificada.

No push da main, o pipeline publica no ECR usando OIDC e atualiza a tag no GitOps. É o ArgoCD que reconcilia essa configuração no cluster.

FAÇA · ONDE → O QUE APARECE

Aba 2 · aponte terraform/ (base), terraform/cluster e terraform/k8s
→ *as três pastas*

Aba 1 · seção "Terraform e ArgoCD" do README
→ *"O backend guarda o state, não o código"*

Aba 3 · grafo de jobs; clique em SAST (gosec) e SCA (Trivy fs)
→ *jobs verdes*

Aba 3 · clique em #1 no título do run → Checks
→ *"Imagem Docker e Trivy (sem push)" verde*

Aba 3 · volte; job "Imagem Docker e push no ECR" → passos Trivy, OIDC e Push; depois "Atualizar tag no GitOps"
→ *commit chore(gitops): auth-service para v1.0.0-…*

03 Prove EKS + GitOps

03:00–05:00 · Lens · Nodes/Pods + ArgoCD

★ OBRIGATÓRIO

DIGA

Este é o cluster EKS provisionado por código.

No Lens, vemos os nós e os pods das aplicações e da observabilidade. O Lens é apenas uma interface de inspeção.

No ArgoCD, as seis aplicações estão sincronizadas com o estado desejado no Git e saudáveis nesta consulta.

A aplicação observability organiza os componentes: kube-prometheus-stack, Loki, Collector gateway e Collector agents.

Os valores dos charts e as configurações do dashboard e dos alertas estão versionados. Assim, a stack de monitoramento também segue GitOps.

FAÇA · ONDE → O QUE APARECE

Lens · tela Nodes
→ *2 nós Ready, versão v1.34*

Lens · Workloads → Pods; filtre togglemaster e depois observability
→ *tudo Running*

Aba 4 · Applications (opcional antes: Terminal C, comandos abaixo)
→ *6 cartões Synced e Healthy*

Aba 4 · clique em observability
→ *árvore com as 4 Applications filhas*

Aba 2 · gitops/observability/values e gitops/observability/grafana
→ *values, dashboard e alertas*

Terminal C

```
kubectl get nodes
```

Terminal C

```
kubectl get applications -n argocd
```

04 Leia a saúde no Grafana

05:00–08:00 · Grafana · ToggleMaster - Visao geral

★ OBRIGATÓRIO

PRINT 01

DIGA

Este dashboard foi customizado para o ToggleMaster. A taxa de requisições mostra o tráfego recebido pelos serviços HTTP.

A proporção de erros indica quantas respostas falharam, e a latência p95 ajuda a observar a experiência nas requisições mais lentas.

Mais abaixo, CPU e memória permitem relacionar comportamento da aplicação e recursos do cluster.

O painel de logs consulta o Loki. Analytics processa eventos de forma assíncrona, portanto não deve ser apresentado como se tivesse o mesmo tráfego HTTP dos demais serviços.

Vou conferir o intervalo de tempo e a atualização para demonstrar dados atuais.

FAÇA · ONDE → O QUE APARECE

Aba 5 · painel "Requisicoes por segundo por servico"
→ *uma linha por serviço HTTP*

Aba 5 · "flag-service: erros 5xx agora", "Taxa de erro (5xx)" e "Latencia p95 por servico"
→ *0% de erro; p95 em ms*

Aba 5 · role até CPU/Memoria por no e por pod
→ *2 nós e os pods do togglemaster*

Aba 5 · "Logs da aplicacao (Loki)"; volte ao painel de requisições
→ *logs recentes; analytics sem linha HTTP*

Aba 5 · canto superior direito
→ *Last 15 minutes e 10s*

05 Conecte logs, métricas e traces

08:00–10:00 · Grafana Explore · Loki + diagrama OTel

★ OBRIGATÓRIO

DIGA

Métricas resumem valores ao longo do tempo; logs registram eventos; traces mostram o caminho de uma execução, dividido em spans. Aqui consulto logs centralizados pelo Loki.

O trace_id permite correlacionar um evento ao rastreamento da requisição.

Em cada nó, um Collector agent lê os logs dos contêineres e os envia ao gateway. As aplicações enviam seus traces ao mesmo gateway.

Ele processa os dados, encaminha logs ao Loki, traces ao New Relic e gera métricas de aplicação pelo spanmetrics para o Prometheus.

As métricas de infraestrutura são coletadas diretamente pelo Prometheus nos exporters.

FAÇA · ONDE → O QUE APARECE

Aba 6 · fonte Loki → Code → cole a consulta 1 → Run query
→ *linhas com "sqs.published"*

Aba 6 · abra uma linha (consulta 2 mostra o flag-service)
→ *service.name, horário e trace_id*

Aba 1 · diagrama do README (ou "Pods, nós e distribuição do Collector")
→ *agents → gateway*

Aba 2 · gitops/observability/values/otel-gateway.yaml, seção pipelines
→ *traces, metrics e logs*

Lens · Pods → observability
→ *node-exporter e kube-state-metrics*

Consulta 1 · Loki

```
{k8s_namespace_name="togglemaster",
service_name="evaluation-service"} |=
"trace_id"
```

Consulta 2 · Loki

```
{k8s_namespace_name="togglemaster",
service_name="flag-service"}
```

06 Siga uma requisição no APM

10:00–13:00 · New Relic · Maps + Distributed tracing

★ OBRIGATÓRIO | PRINT 02

DIGA

Escolhemos New Relic como o APM comercial desta entrega. Ele recebe os traces via OpenTelemetry. No mapa, observamos as dependências dos cinco microsserviços.

Agora faço uma requisição ao evaluation-service.

No trace detalhado, vemos as chamadas aos serviços de flags, segmentação e autenticação, além das dependências de dados.

O caminho varia: um acerto no Redis pode encurtar a execução. Para a demonstração completa, seleciono uma requisição sem cache e confiro seu horário.

O fluxo de SQS e analytics é assíncrono; a resposta HTTP não espera a persistência analítica. A duração de cada span ajuda a localizar uma operação lenta.

FAÇA · ONDE → O QUE APARECE

Aba 8 · Service map; amplie as dependências se precisar
→ *os cinco serviços ligados*

Terminal C · comandos ① e ② (próximo slide)
→ *"Flag criada: video-…" e "result":true*

Aba 9 · filtre evaluation-service → GET /evaluate → trace do horário do curl
→ *cascata com os serviços*

Aba 9 · aponte o span do Redis e o horário
→ *o trace da flag nova*

Aba 9 · span da SQS; clique no span mais longo
→ *duração e atributos*

Cena 06 · comandos do Terminal C

A chave fica dentro do pod trafego e não aparece na tela.

① cria uma flag nova (o cache é por nome da flag)

→ Flag criada: video-…

```
kubectl exec -n togglemaster trafego -- sh -c 'NOME="video-$(date +%s)"; curl --fail-with-body -s -X POST http://flag-service:8002/flags -H "Authorization: Bearer $CHAVE" -H "Content-Type: application/json" -d "{\"name\":\"$NOME\",\"description\":\"Demonstracao Fase 4\",\"is_enabled\":true}" >/dev/null && curl --fail-with-body -s -X POST http://targeting-service:8003/rules -H "Authorization: Bearer $CHAVE" -H "Content-Type: application/json" -d "{\"flag_name\":\"$NOME\",\"is_enabled\":true,\"rules\":{\"type\":\"PERCENTAGE\",\"value\":100}}" >/dev/null && echo "Flag criada: $NOME"'
```

② requisição de demonstração: troque NOME_DA_FLAG pelo nome impresso

→ "result":true; o trace aparece no New Relic em 10 a 60 s

```
curl -s "http://127.0.0.1:8004/evaluate?flag_name=NOME_DA_FLAG&user_id=video-1"; echo
```

07 Provoque a falha controlada

13:00–15:00 · Terminal · tráfego + flag-service

★ OBRIGATÓRIO

DIGA

Agora vou provocar uma falha transitória controlada no flag-service. O objetivo é demonstrar detecção e reação automáticas, sem desligar um banco.

A rota administrativa exige autenticação e faz as rotas de flags responderem 503.

O health check continua saudável; por isso, o Kubernetes não reinicia o processo apenas pela liveness probe.

O tráfego contínuo permite medir a proporção de erros.

A falha fica na memória do processo e desaparece quando um novo pod assume. Vou observar a cadeia sem clicar em Run workflow nem reiniciar manualmente o serviço.

FAÇA · ONDE → O QUE APARECE

Terminal A · aponte o log
→ *o log mostra -> 200*

Terminal C · comando ③ (próximo slide)
→ *Terminal A passa a -> 503*

Terminal C · comando ④; aponte o Terminal B
→ *health: 200; pod Running, 0 restarts*

Aba 5 · "flag-service: erros 5xx agora"
→ *sobe em 30 a 60 s*

Aba 7 · tire as mãos do teclado e vá para Alert rules

Cena 07 · comandos do Terminal C

Depois do ③, não reinicie nada: a correção precisa ser automática.

③ ativa a falha controlada

→ Terminal A passa a -> 503

```
kubectl exec -n togglemaster trafego -- sh -c 'curl --fail-with-body -s -X POST
http://flag-service:8002/admin/fault -H "Authorization: Bearer $CHAVE" -H "Content-
Type: application/json" -d "{\"enabled\":true}"'; echo
```

④ prova que o health continua saudável

→ health: 200

```
kubectl exec -n togglemaster trafego -- curl -s -o /dev/null -w "health:
%{http_code}\n" http://flag-service:8002/health
```

08 Acompanhe alerta e comunicação

15:00–17:00 · Grafana Alerting → PagerDuty → Slack

★ OBRIGATÓRIO

PRINT 03

DIGA

A regra avalia a proporção de erros 5xx do flag-service.

O limite escolhido é maior que dez por cento durante um minuto; isso evita reagir a um único erro isolado. O exemplo do enunciado usa outro serviço e outro percentual, mas nossa regra atende ao mesmo objetivo.

Quando entra em Firing, o Grafana abre um incidente no PagerDuty e envia o contexto ao Slack.

PagerDuty organiza o incidente e o acionamento humano; Slack concentra a comunicação.

Aqui confiro serviço, horário e descrição para mostrar que as telas correspondem à mesma falha. Um DatasourceError não é a evidência desta regra de erros HTTP.

FAÇA · ONDE → O QUE APARECE

Aba 7 · FlagService5xxAlto
→ *Pending 30 a 60 s depois da falha*

Aba 7 · espere
→ *Firing cerca de 1 min depois*

Aba 10 · incidente novo → abra os detalhes
→ *Triggered, criado sozinho*

Aba 11 · #incidentes-tech-challenge
→ *[FIRING:1] FlagService5xxAlto e cartão do PagerDuty*

Aba 11 · aponte Servico, Detalhe e Inicio
→ *mesmo horário do Firing*

Atenção Não leia a linha "O que aconteceu" do Slack: ela mostra o texto do modelo no lugar do percentual.

09 Prove a correção automática

17:00–20:00 · GitHub Actions · Self-healing + terminal

★ OBRIGATÓRIO

PRINT 04

DIGA

O mesmo alerta chama o webhook do GitHub e dispara esta execução automaticamente.

O workflow assume uma role temporária por OIDC.

Antes de agir, o script valida o serviço em uma lista permitida e verifica um intervalo mínimo de dez minutos entre reinícios. Ele executa rollout restart e aguarda o rollout terminar.

Aqui vemos o registro da ação e a substituição do pod.

O tráfego volta a 200, o alerta retorna ao normal e o incidente é resolvido.

Isso é uma tentativa de mitigação, não uma cura universal: um banco indisponível, por exemplo, não seria corrigido apenas reiniciando a aplicação.

FAÇA · ONDE → O QUE APARECE

Aba 12 · atualize a página
→ *run novo: repository_dispatch*

Aba 12 · run → job → "Assumir a role do self-healing (OIDC)"
→ *passo verde*

Aba 12 · expanda "Reiniciar com allowlist e cooldown"
→ *allowlist, cooldown e "restarted"*

Terminal B · aponte a troca
→ *antigo Terminating, novo Running*

Terminal A · depois abas 7, 10 e 11
→ *log em -> 200; alerta Normal; incidente Resolved*

Só fala · mantenha o Terminal A na tela

10 Feche com as evidências

20:00–21:00 · README + relatório PDF

DIGA

Demonstramos a base automatizada, as métricas e os logs centralizados, o roteamento pelo Collector e o tracing no APM. Na prova do incidente, acompanhamos a falha controlada, o alerta, a comunicação e a tentativa automática de recuperação.

O repositório reúne o código de infraestrutura, a instrumentação e a automação.

O relatório traz os quatro prints exigidos e justifica a arquitetura e as escolhas de New Relic e PagerDuty.

As evidências precisam corresponder ao que foi realmente executado. Obrigado.

FAÇA · ONDE → O QUE APARECE

Aba 1 · segundo diagrama de "Arquitetura de observabilidade e resposta"
→ *Grafana → PagerDuty, Slack e Actions*

Aba 1 · "Onde está cada requisito da Fase 4"
→ *requisito → arquivo*

PDF local · docs/relatorio/RELATORIO_ENTREGA_2026-11-10_v02.pdf
→ *prints e justificativas*

STOP · pare a gravação
→ *duração até 25 min*

Se algo der errado

Não reinicie à mão como se fosse automático; diga no vídeo quando houver intervenção.

Túnel não responde	tail -5 /tmp/tunel-grafana.log (troque o nome); porta ocupada: feche o Docker Desktop
Alerta não dispara	Terminal A mostra -> 503? o painel de 5xx subiu? espere de 1 a 3 min
Self-healing termina em cooldown	faz menos de 10 min do último restart: espere e repita
Trace não aparece	espere 10 a 60 s; aumente o período para Last 30 minutes
Pod com erro	kubectl describe pod <pod> -n <namespace> | tail -20 e kubectl logs <pod> -n <namespace>

Contingência: só se a automação falhar; desliga a falha e não conta como self-healing

```
kubectl exec -n togglemaster trafego -- sh -c 'curl --fail-with-body -s -X POST http://flag-service:8002/admin/fault -H
"Authorization: Bearer $CHAVE" -H "Content-Type: application/json" -d "{\"enabled\":false}"'; echo
```

Cola rápida · números e conceitos

Para responder sem consultar outro documento.

5	2	6	>10% · 1 min	600 s
microsserviços	nós Ready	Applications	regra de 5xx	cooldown

Erro 5xx — o pedido estava certo, mas o servidor falhou (500 erro interno; 503 indisponível, o da falha controlada). 4xx é erro de quem chamou, como o 401 de chave inválida, e não entra no alerta.

Latência p95 — 95% das requisições responderam nesse tempo ou menos; 5% demoraram mais. A média esconde os lentos, e o p95 os mostra.

RED — Rate, Errors, Duration: requisições/s, erros 5xx e p95, os três painéis do topo do dashboard.

Self-healing — Grafana → repository_dispatch → GitHub Actions (OIDC) → lista permitida + cooldown → rollout restart.

Depois do STOP

Confira vídeo e prints antes de qualquer destroy.

PRINT 01 Grafana · dashboard com dados (pode ser em 2 imagens)

PRINT 02 New Relic · trace aberto com spans e durações

PRINT 03 Slack · mensagem do incidente

PRINT 04 Actions · passo do restart + Terminal B com o pod novo

- Salve os prints em docs/evidencias/entrada-local/ e cubra chaves e e-mails.
- Apoios úteis: Service Map, Firing, PagerDuty, Loki, ArgoCD e Lens.
- Feche o Terminal D para encerrar os túneis; taskkill //IM kubectl.exe //F só no fim.
- O encerramento da AWS segue o guia de deploy (Parte 4), com plano salvo.